Backend Storage & Query Migration to OpenSearch

Technical Alignment Document for UI and Backend Teams

Date: March 19, 2026
Prepared by: OpenSearch UI Engineering Manager
Audience: Backend Engineering Team

Executive Summary

The backend team is reimplementing storage and query layers on OpenSearch infrastructure. This document outlines critical alignment points, technical requirements, and execution strategy to ensure **zero frontend impact** during this migration.

Core Principle: Backend changes must be transparent to the UI layer. All existing OSD API contracts must remain stable.

---

1. Critical Alignment Points

1.1 API Contract Stability (Non-Negotiable)

Backend Team Must Guarantee:

Aspect	Requirement	Rationale
HTTP Endpoints	No changes to paths, methods, or URL parameters	Frontend has hardcoded endpoint references
Request Schema	Identical request body structure, query params, headers	Client-side validation and serialization depends on current schema
Response Schema	Same JSON structure, field names, data types	Frontend parsers and UI components expect exact field mappings
Error Responses	Maintain existing error codes and formats	Error handling logic relies on specific error structures
Authentication/Authorization	No changes to auth flow or headers	Session management and security interceptors are UI-owned

Verification Method: OpenAPI spec diff analysis (use openapi-spec.yaml as baseline)

1.2 API Performance Characteristics

Backend Must Maintain or Improve:

- **Response Times:** No regression beyond 10% of current p95 latency
- **Pagination:** Existing page size limits and cursor behavior must remain identical
- **Rate Limits:** If introducing new rate limits, they must be communicated 2 sprints in advance
- **Timeouts:** Maintain current timeout thresholds (or provide configuration to adjust)

Critical APIs (from analysis):

- /api/saved_objects/_find (12 usages across UI)
- /internal/search/{strategy} (core search functionality)
- /api/saved_objects/_bulk_get (7 usages)
- /api/saved_objects/{type}/{id} (CRUD operations)

1.3 Functional Parity Requirements

Zero Behavioral Changes for:

1. **Saved Objects Operations**
    - CRUD operations (create, read, update, delete)
    - Bulk operations (bulk create, bulk update)
    - Import/export workflows (NDJSON format)
    - Conflict resolution on import
    - Namespace isolation (multi-tenancy)
2. **Search & Query**
    - OpenSearch query DSL passthrough (Console proxy)
    - Search strategies (default, async, DQL, PPL)
    - Aggregations and bucketing
    - Scroll API behavior
    - Multi-search batching
3. **Data Source Management**
    - Multi-cluster connectivity
    - Connection pooling behavior
    - Data source switching in UI
4. **Workspace & Permissions**
    - Workspace isolation
    - Object-level permissions
    - Feature flags and capabilities resolution

---

2. Technical Proposal

2.1 Backend Implementation Strategy

Recommended Approach: Facade Pattern with Feature Flags

```
┌─────────────────────────────────────────────────────────────────┐
│         OSD HTTP API Layer (Unchanged)        │
│   /api/saved_objects, /internal/search, etc.   │
└───────────────────────────┬─────────────────────────────────────┘
            │
            v
┌─────────────────────────────────────────────────────────────────┐
│       Service Facade (Abstraction Layer)      │
│   Feature Flag: USE_OPENSEARCH_BACKEND=true    │
└───────────────────┬───────────────────────────┬─────────────────┘
        │                  │
        v                  v
  ┌──────────────────────┐   ┌─────────────────────────────────┐
  │  Legacy        │      │  OpenSearch        │
  │  Backend       │      │   Backend (New)       │
  └──────────────────────┘   └─────────────────────────────────┘
```

Benefits:

- Gradual rollout with instant rollback capability
- A/B testing in production
- Parallel validation of old vs. new backend
- Zero downtime migration

2.2 Implementation Phases

Phase 1: Foundation (Weeks 1-2)

- Implement facade layer with feature flags
- Set up OpenSearch backend infrastructure
- Implement core Saved Objects repository on OpenSearch
- Shadow mode: Dual-write to both backends, read from legacy

Phase 2: Validation (Weeks 3-4)

- Enable read traffic from OpenSearch backend (internal testing)
- Response diff validation: Compare legacy vs. OpenSearch responses
- Performance benchmarking: Ensure no regression
- Integration testing: Run full Cypress suite against OpenSearch backend

Phase 3: Gradual Rollout (Weeks 5-6)

- 10% traffic to OpenSearch backend (canary deployment)
- Monitor error rates, latency, data consistency
- 50% → 100% rollout if metrics are green
- Deprecate legacy backend

Phase 4: Cleanup (Week 7+)

- Remove facade layer (optional)

- Remove legacy backend code
- Documentation updates

2.3 Technical Safeguards

Mandatory Before Production Rollout:

1. **API Contract Tests**
    - OpenAPI spec validation: All responses must match openapi-spec.yaml
    - JSON Schema validation for all 96 OSD API endpoints
    - Automated regression tests run on every PR
2. **Integration Test Coverage**
    - All 120+ OSD API endpoints tested end-to-end
    - Cypress test suite: 100% pass rate required
    - Saved Objects import/export workflows validated
    - Multi-data-source scenarios tested
3. **Performance Benchmarks**
    - Baseline: Current p50, p95, p99 latencies for all critical APIs
    - Target: No regression > 10% at p95
    - Load testing: 2x peak traffic simulation
4. **Data Migration & Consistency**
    - Data migration script with rollback capability
    - Checksum validation: All migrated data must match source
    - No data loss tolerance: 100% data integrity required

---

3. Execution Strategy

3.1 Team Responsibilities

Team	Responsibilities	Deliverables
Backend	• OpenSearch backend implementation • API contract compliance • Data migration tooling • Performance optimization	• OpenSearch repository implementation • Migration scripts • Performance benchmarks • Rollback procedures
UI (Our Team)	• API contract verification • Integration test execution • UAT coordination • Rollback decision authority	• OpenAPI spec baseline • Integration test results • Go/no-go decision • Production monitoring
QA	• End-to-end testing • Performance validation • UAT test plan execution	• Test results • Sign-off for production

3.2 Communication Cadence

- **Weekly Sync:** Progress updates, blocker escalation
- **Pre-Rollout Review:** Go/no-go decision 48 hours before each phase
- **Daily Standups (during rollout):** Quick sync during Phase 3
- **Incident Response:** Backend team on-call during rollout windows

3.3 Success Criteria (Go/No-Go Gates)

Phase 2 → Phase 3 (Validation → Rollout):

- 100% API contract compliance (all 96 endpoints)
- 100% Cypress test pass rate
- Zero data consistency issues in shadow mode
- Performance: p95 latency within 10% of baseline
- Zero critical bugs in internal testing

Phase 3 Milestones (Gradual Rollout):

- 10% canary: Error rate < 0.1%, latency within 10% baseline
- 50% rollout: Sustained performance for 48 hours, no incidents
- 100% rollout: Final sign-off from UI, QA, and Backend leads

3.4 Rollback Strategy

Immediate Rollback Triggers:

- Error rate increase > 1% absolute
- p95 latency increase > 25%
- Data loss or corruption detected
- Critical bug affecting user workflows

Rollback Procedure:

- Feature flag flip: USE_OPENSEARCH_BACKEND=false (30 seconds)
- Traffic routes back to legacy backend automatically
- Post-mortem within 24 hours
- Fix-forward plan or extended testing period

---

4. Risk Mitigation

4.1 Identified Risks

Risk	Impact	Mitigation
API response schema drift	HIGH	OpenAPI spec validation in CI/CD pipeline
Performance regression	HIGH	Benchmark gates in each phase, canary deployment
Data migration errors	CRITICAL	Dry-run migrations, checksum validation, rollback scripts
Edge case handling differences	MEDIUM	Comprehensive integration test coverage (Cypress suite)
Multi-data-source issues	MEDIUM	Dedicated testing for MDS scenarios

4.2 Validation Checklist

Before Each Rollout Phase:

- [ ] All API endpoints tested (automated)
- [ ] Cypress suite: 100% pass rate
- [ ] Performance benchmarks: Green
- [ ] Data consistency validation: Passed
- [ ] Rollback procedure tested and documented

- [ ] On-call team briefed and ready

---

5. UI Team Requirements from Backend

5.1 Deliverables Needed

1. **Updated OpenAPI Spec (if any internal changes)**
    - Provide diff report showing zero breaking changes
    - Due: 1 week before Phase 3 rollout
2. **Data Migration Plan**
    - Detailed steps, rollback procedure, timeline
    - Due: Before Phase 1 starts
3. **Performance Test Results**
    - Baseline vs. OpenSearch backend comparison
    - Due: End of Phase 2
4. **Feature Flag Configuration**
    - Documentation on how to toggle backends
    - Due: Phase 1 completion

5.2 Access Requests

- [ ] Read-only access to OpenSearch backend logs (for debugging)
- [ ] Metrics dashboard for backend performance monitoring
- [ ] Canary deployment control (ability to adjust traffic percentage)

---

6. Next Steps

Immediate Actions (This Week):

1. Backend team: Confirm alignment on API contract stability requirements
2. Both teams: Review and approve this technical proposal
3. Backend team: Provide detailed implementation plan with timeline
4. UI team: Prepare OpenAPI spec baseline and integration test suite

Ongoing:

- Weekly syncs starting next week
- Shared Slack channel: #backend-migration-2026
- Shared documentation: Confluence page with runbooks

Appendix: Reference Materials

- **OpenAPI Spec Baseline:** mustang/openapi-spec.yaml
- **API Mapping:** mustang/OSD_TO_OPENSEARCH_API_MAPPING.pdf

- **Team Assignment Tracker:** mustang/OpenSearch_API_Usage_TeamAssignment.xlsx
- **Integration Tests:** cypress/integration/ directory

Contact:

- UI Engineering Manager: [Your Name]
- Backend Engineering Lead: [TBD]
- QA Lead: [TBD]

Document Status: Draft for Review
Next Review Date: [Fill in date for weekly sync]
Approval Required From: Backend Team Lead, QA Lead, Product Manager